

01 — Run and debug Python scripts in VS Code / 在 VS Code 中运行与调试 Python 脚本

Question and example / 问题与示例

How do I run the open Python file, pass options, and pause inside a function to inspect its values? This first edition uses the existing [chapter 10 script](#). It needs Python 3.9+ and only the standard library. It prints values and does not create files. The original Spyder tutorial is now [01-SI — Spyder](#), supplementary information for this chapter.

怎样运行当前打开的 Python 文件、传入选项，并在函数内部暂停查看变量？本版使用已有的第 10 章脚本，需要 Python 3.9+，只用标准库；它打印数值，不创建文件。原 Spyder 教程移至 [01-SI — Spyder](#)，作为本章补充资料（Supplementary Information）。

1. Choose the interpreter / 选择解释器

Open the repository folder in VS Code. Confirm that Microsoft's Python and Python Debugger extensions are available. An editor displays your code; the Python interpreter executes it.

在 VS Code 中打开仓库文件夹，确认 Microsoft 的 Python 与 Python Debugger 扩展可用。编辑器（editor）显示代码，Python 解释器（interpreter）负责执行代码。

Press **Ctrl+Shift+P**, run **Python: Select Interpreter**, and choose an installed environment. The selected interpreter is normally used for Python runs and debugging; a debug configuration can override it. See [VS Code: environments](#) ^[1].

按 **Ctrl+Shift+P**，执行 **Python: Select Interpreter**，选择已有环境。Python 的运行和调试通常使用该解释器，但调试配置可以指定另一个解释器。参见 [VS Code: 环境](#) ^[1]。

In a newly opened terminal, check which interpreter the command `python` actually resolves to:

新建终端后，检查 `python` 命令实际指向哪个解释器：

```
python -c "import sys; print(sys.executable); print(sys.version)"
```

Compare the printed executable with your selection. An already-open terminal can still use a previous environment. For environment setup, see [chapter 07](#).

将输出的可执行文件路径与所选解释器比较。此前打开的终端可能仍使用旧环境；环境设置见第 07 章。

2. Run the open file / 运行当前文件

Open `examples/10_python_annotations_and_parsing.py`, save it, and choose **Run Python File in Terminal** from the upper-right play button. Inspect the command and output in **Terminal**. If another extension supplies a similar triangle, identify the Python command by its label. See [VS Code: running Python](#) ^[2].

打开并保存 `examples/10_python_annotations_and_parsing.py`，在右上角三角按钮中选择 **Run Python File in Terminal**，在 **Terminal** 查看实际命令与输出。其他扩展也可能提供三角按钮，要根据命令名称确认。参见 [VS Code: 运行 Python](#) ^[2]。

Running executes the script to completion unless it waits for input or encounters an error. A breakpoint only pauses execution when a debugger is attached. For this example, running without options produces:

运行（Run）会执行脚本，直到结束、等待输入或遇到错误。断点（breakpoint）需要在连接调试器时才会暂停程序。本例不带选项运行时输出：

```
p value: {'rho': 50.0}
p type: dict
annotation: dict[str, float] | None
annotation type: str
output path: None
```

None here means no output path was supplied; it is not an error. The meanings of the annotation and `__name__` guard are explained in [chapter 10](#).

这里的 None 表示没有传入输出路径，不是报错。类型标注与 `__name__` 入口判断见第 10 章。

3. Run from the terminal and pass an option / 在终端运行并传入选项

At a PowerShell prompt, with the repository root as the current directory, run:

在 PowerShell 提示符下，以仓库根目录为当前目录，执行：

```
Get-Location
python examples/10_python_annotations_and_parsing.py
python examples/10_python_annotations_and_parsing.py --output-dir results/demo
```

The second Python command changes the final output to `output path: results/demo`. In this teaching script the path is parsed and printed; no directory is created. `python` selects the interpreter, the `.py` path selects the script, and `--output-dir results/demo` supplies a script option.

第二条 Python 命令将最后一行输出改为 `output path: results/demo`。这个教学脚本只解析并打印路径，不创建目录。`python` 选择解释器，`.py` 路径选择脚本，`--output-dir results/demo` 向脚本传入选项。

The current working directory determines how relative paths are interpreted. It is not necessarily the script's folder. A prompt beginning with `>>>` is a Python REPL; enter `exit()` before typing these PowerShell commands.

当前工作目录（current working directory）决定相对路径的起点，不一定是脚本所在文件夹。如果提示符是 `>>>`，当前处于 Python 交互环境（REPL）；先输入 `exit()`，再执行这些 PowerShell 命令。

4. Pause and inspect a function / 暂停并观察函数

Keep the example `.py` file active. Click the gutter next to `value = identity(p)` to add a breakpoint. Start debugging with `F5`; if prompted, choose **Python Debugger** and the current Python file. If an existing launch configuration targets another program, choose the current-file configuration instead. See [Python debugging](#) ^[3].

保持示例 `.py` 文件为当前编辑文件。在 `value = identity(p)` 行号旁的空白处点击，添加断点。按 `F5` 开始调试；如出现选择提示，选择 **Python Debugger** 和当前 Python 文件。若已有启动配置指向其他程序，应改选当前文件配置。参见 [Python 调试](#) ^[3]。

At the breakpoint, the highlighted statement has not executed yet. On this fresh run, `p` is `{'rho': 50.0}`, while `value` has not been assigned. Press **Step Into** (F11) to enter `identity`. At return `p`, its local parameter `p` refers to that same dictionary.

停在断点时，高亮语句还未执行。在本次新启动的运行中，`p` 已经是 `{'rho': 50.0}`，但 `value` 尚未赋值。按 **Step Into / 单步进入** (F11)，进入 `identity`；停在 `return p` 时，函数的局部参数 `p` 指向同一个字典。

Inspect `p`, `type(p)`, and `p["rho"]` in **Debug Console** while paused; the results are the dictionary, `dict`, and `50.0`. After returning to `main`, use Step Over if needed until the assignment has finished. Then `value` is available and `value is p` is `True`: this function returns the original object, not a copy.

暂停时在 **Debug Console / 调试控制台** 中查看 `p`、`type(p)`、`p["rho"]`，分别得到字典、`dict` 类型和 `50.0`。返回 `main` 后，如赋值尚未完成，再执行 Step Over。完成后可查看 `value`，且 `value is p` 为 `True`：函数返回的是原对象，没有复制字典。

Windows default action Windows 默认操作	What it does 作用
F10 — Step Over 单步跳过	Execute the current line without entering its function calls; other breakpoints can still interrupt. 执行当前行，不逐行进入其中的函数；其他断点仍可能使程序暂停。
F11 — Step Into 单步进入	Enter a called function when supported by the debugger. 在调试器支持时进入被调用函数。
Shift+F11 — Step Out 单步跳出	Finish the current function and return to its caller. 执行完当前函数，返回调用它的位置。
F5 — Continue 继续	Resume until another breakpoint, a configured exception stop, or completion. 继续到下一断点、已配置的异常暂停或程序结束。
Shift+F5 — Stop 停止	End the debugging session. 结束本次调试。

Variables, **Watch**, and **Call Stack** help inspect the paused state. The selected stack frame determines the available local variables. See [VS Code: debugging controls](#) ^[4]. Shortcuts can be customized; use toolbar labels if yours differ.

通过 **Variables / 变量**、**Watch / 监视** 和 **Call Stack / 调用堆栈** 查看暂停状态。选中的栈帧（stack frame）决定可查看哪些局部变量。参见 [VS Code: 调试控制](#) ^[4]。快捷键可能被自定义，可按工具栏名称操作。

5. Keep the three consoles distinct / 区分三种输入位置

Location 位置	What to enter 输入内容
Terminal with a PowerShell prompt 显示 PowerShell 提示符的终端	Shell commands, including <code>python script.py</code> . Shell 命令，例如 <code>python script.py</code> 。
Python REPL with <code>>>></code> 显示 <code>>>></code> 的 Python 交互环境	Python statements, in that REPL's own session. Python 语句，使用该交互会话自己的变量。

Location 位置	What to enter 输入内容
Debug Console during a paused session 调试暂停时的 Debug Console	Expressions for the selected frame, such as p["rho"]. 当前选中栈帧中的表达式，例如 p["rho"]。

A separately launched script's local variables do not automatically appear in a different Python REPL after it exits. Pause before the function returns to inspect them.

独立启动的脚本退出后，它的局部变量不会自动出现在另一个 Python REPL 中。要观察这些变量，应在函数返回前暂停。

If the file is not found, check the working directory and script path. If a module is missing, check the interpreter and its dependencies. If a breakpoint does not stop, check that you started debugging, selected the correct script, and reached that executable line.

找不到文件时，检查工作目录与脚本路径；缺少模块时，检查解释器及其依赖；断点不停时，检查是否启动调试、是否选中正确脚本，以及是否执行到该可执行语句。

Verification and next discussion / 核对与后续讨论

The two terminal runs above were executed with Python 3.13.9 and produced the displayed values. The VS Code steps were checked against official documentation; the graphical debug session has not been performed here.

上述两种终端运行已在 Python 3.13.9 下执行，得到所列输出。VS Code 操作已核对官方文档；本次未实际操作图形界面完成调试会话。

Next, we can work through your own pause-and-step session, then add saved arguments in launch.json, exception breakpoints, and traceback reading as needed. This chapter starts with the basic run/debug workflow; 01-SI preserves the more detailed Spyder array-inspection example.

接下来可以结合你的实际操作，逐步讨论暂停与单步执行，再按需要补充 launch.json 中保存运行参数、异常断点，以及错误回溯（traceback）的阅读方法。本章先建立基本运行与调试流程；01-SI保留详细的 Spyder 数组检查案例。

参考资料 (References)

1. <https://code.visualstudio.com/docs/python/environments>
2. <https://code.visualstudio.com/docs/python/run>
3. <https://code.visualstudio.com/docs/python/debugging>
4. <https://code.visualstudio.com/docs/debugtest/debugging>